PROYECTO 1: CryptoFolio API

📋 Índice

1. Descripción del Proyecto
2. Objetivos de Aprendizaje
3. Stack Técnico Oficial
4. Arquitectura del Proyecto
5. Roadmap de Tickets
6. Setup Rápido Inicial

📖 Descripción del Proyecto

CryptoFolio es una API REST para gestión de portafolios de criptomonedas que permite a los usuarios crear múltiples carteras, registrar transacciones (compra/venta), y calcular automáticamente el balance y rendimiento de sus inversiones.

🎯 Problema que resuelve

Los inversores en criptomonedas necesitan trackear sus compras/ventas en múltiples exchanges y calcular manualmente su rendimiento. CryptoFolio centraliza toda esta información y automatiza los cálculos.

✨ Funcionalidades Core (MVP)

1. **Gestión de usuarios**: Registro, login con JWT
2. **Gestión de portfolios**: CRUD de carteras (un usuario puede tener múltiples portfolios)
3. **Gestión de transacciones**: Añadir compras/ventas de crypto
4. **Cálculo automático**: Balance actual, P&L (Profit & Loss), ROI por portfolio
5. **Histórico**: Ver todas las transacciones de un portfolio

🎓 Lo que aprenderás

- **Clean Architecture** aplicada desde día 1
- **Domain-Driven Design** (entidades, value objects, agregados)
- **Spring Security** con JWT
- **Testing profesional** (unitarios, integración, E2E)
- **Manejo de errores** centralizado
- **Validación de datos** con Bean Validation
- **Documentación automática** con OpenAPI/Swagger

🎯 Objetivos de Aprendizaje

Técnicos

- ✅ Implementar Clean Architecture con separación estricta de capas
- ✅ Aplicar principios SOLID en diseño de clases
- ✅ Gestionar autenticación/autorización con Spring Security + JWT
- ✅ Diseñar modelo de datos relacional optimizado
- ✅ Escribir tests con alta cobertura (>75%)
- ✅ Dockerizar aplicación completa

Arquitectónicos

- ✅ Entender inversión de dependencias (Domain no depende de nada)
- ✅ Separar lógica de negocio de framework
- ✅ Diseñar APIs REST siguiendo convenciones
- ✅ Implementar DTOs para desacoplar domain de API

Profesionales

- ✅ Estructura de proyecto enterprise-ready
- ✅ Documentación técnica clara (README, ADRs)
- ✅ Control de versiones con Git Flow

- ✅ CI/CD básico (GitHub Actions)

🛠️ Stack Técnico Oficial

Backend

```
Lenguaje: Java 17 (LTS)
Framework: Spring Boot 3.2.x
Build Tool: Maven 3.9.x

Spring Modules:
  - Spring Web (REST APIs)
  - Spring Security (Auth/JWT)
  - Spring Data JPA (ORM)
  - Spring Validation (Bean Validation)
  - Spring Actuator (Health checks, metrics)

Base de Datos:
  - PostgreSQL 16 (development & production)
  - H2 (in-memory para tests)

Testing:
  - JUnit 5 (framework de testing)
  - Mockito (mocking)
  - AssertJ (assertions fluidas)
  - Testcontainers (integration tests con PostgreSQL real)
  - REST Assured (E2E API tests)

Documentación:
  - SpringDoc OpenAPI 3 (Swagger UI)

Logging:
  - SLF4J + Logback

Utilidades:
  - Lombok (reducir boilerplate)
  - MapStruct (mapeo DTO ↔ Entity)
  - Flyway (migraciones de DB)

DevOps:
  - Docker + Docker Compose
  - GitHub Actions (CI)
```

Frontend (Opcional - Fase 2)

```
Lenguaje: TypeScript 5.x
Framework: React 18.x
Build Tool: Vite 5.x

Librerías UI:
  - Tailwind CSS (estilos utility-first)
  - shadcn/ui (componentes accesibles)
  - Recharts (gráficos de rendimiento)

Estado:
  - TanStack Query (manejo de server state)
  - Zustand (client state management)

Routing:
  - React Router v6

Formularios:
  - React Hook Form + Zod (validación)

Testing:
  - Vitest (unit tests)
  - React Testing Library
  - Playwright (E2E)
```

Infraestructura

```
Contenedores: Docker 24.x
Orquestación Local: Docker Compose
CI/CD: GitHub Actions
Deployment: Railway / Render (backend), Vercel (frontend)
```

🏗️ Arquitectura del Proyecto

Clean Architecture en Capas

```
com.cryptofolio
│
├── domain/                          # ❤️ CORAZÓN DEL NEGOCIO (sin dependencias e
│   ├── entities/                    # Entidades de negocio
│   │   ├── User.java
```

```
│   │   ├── Portfolio.java
│   │   └── Transaction.java
│   │
│   ├── valueobjects/                # Value Objects (inmutables)
│   │   ├── Money.java
│   │   ├── Crypto.java              # Ej: BTC, ETH (enum + metadata)
│   │   └── TransactionType.java     # BUY, SELL
│   │
│   ├── aggregates/                  # Agregados (raíz: Portfolio)
│   │   └── PortfolioAggregate.java
│   │
│   ├── repositories/                # Interfaces (ports)
│   │   ├── UserRepository.java
│   │   ├── PortfolioRepository.java
│   │   └── TransactionRepository.java
│   │
│   ├── services/                    # Servicios de dominio (lógica pura)
│   │   ├── PortfolioCalculator.java # Calcula balance, P&L, ROI
│   │   └── TransactionValidator.java
│   │
│   └── exceptions/                  # Excepciones de negocio
│       ├── InsufficientFundsException.java
│       └── PortfolioNotFoundException.java
│
├── application/                     # 🎯 CASOS DE USO (orquestación)
│   ├── usecases/
│   │   ├── auth/
│   │   │   ├── RegisterUserUseCase.java
│   │   │   └── LoginUserUseCase.java
│   │   │
│   │   ├── portfolio/
│   │   │   ├── CreatePortfolioUseCase.java
│   │   │   ├── GetPortfolioUseCase.java
│   │   │   ├── UpdatePortfolioUseCase.java
│   │   │   ├── DeletePortfolioUseCase.java
│   │   │   └── ListUserPortfoliosUseCase.java
│   │   │
│   │   └── transaction/
│   │       ├── AddTransactionUseCase.java
│   │       ├── GetTransactionHistoryUseCase.java
│   │       └── DeleteTransactionUseCase.java
│   │
│   ├── dtos/                        # DTOs (contratos de API)
│   │   ├── request/
│   │   │   ├── RegisterUserRequest.java
│   │   │   ├── LoginRequest.java
│   │   │   ├── CreatePortfolioRequest.java
```

```
│   │   │   └── AddTransactionRequest.java
│   │   │
│   │   └── response/
│   │       ├── AuthResponse.java
│   │       ├── PortfolioResponse.java
│   │       ├── PortfolioSummaryResponse.java  # Balance, P&L, ROI
│   │       └── TransactionResponse.java
│   │
│   ├── mappers/                     # Mappers (DTO ↔ Entity)
│   │   ├── UserMapper.java
│   │   ├── PortfolioMapper.java
│   │   └── TransactionMapper.java
│   │
│   └── ports/                       # Interfaces para servicios externos
│       └── CryptoPriceProvider.java # Ej: obtener precio actual de BTC
│
├── infrastructure/                  # 🔧 DETALLES TÉCNICOS (frameworks, DB, APIs
│   ├── controllers/                 # REST Controllers
│   │   ├── AuthController.java
│   │   ├── PortfolioController.java
│   │   └── TransactionController.java
│   │
│   ├── persistence/                 # Implementaciones de repositorios
│   │   ├── entities/                # JPA Entities (diferente de domain entities
│   │   │   ├── UserEntity.java
│   │   │   ├── PortfolioEntity.java
│   │   │   └── TransactionEntity.java
│   │   │
│   │   ├── repositories/            # Spring Data JPA repos
│   │   │   ├── JpaUserRepository.java
│   │   │   ├── JpaPortfolioRepository.java
│   │   │   └── JpaTransactionRepository.java
│   │   │
│   │   └── adapters/                # Adapters (infraestructura → domain)
│   │       ├── UserRepositoryAdapter.java
│   │       ├── PortfolioRepositoryAdapter.java
│   │       └── TransactionRepositoryAdapter.java
│   │
│   ├── security/                    # Spring Security config
│   │   ├── SecurityConfig.java
│   │   ├── JwtTokenProvider.java
│   │   ├── JwtAuthenticationFilter.java
│   │   └── UserDetailsServiceImpl.java
│   │
│   ├── config/                      # Configuración general
│   │   ├── DatabaseConfig.java
│   │   ├── OpenAPIConfig.java
```

```
│   │   └── CorsConfig.java
│   │
│   ├── exception/                  # Manejo global de errores
│   │   ├── GlobalExceptionHandler.java
│   │   └── ErrorResponse.java
│   │
│   └── external/                   # Servicios externos (APIs)
│       └── CoinGeckoPriceProvider.java  # Implementa CryptoPriceProvider
│
└── CryptoFolioApplication.java     # Main class
```

Flujo de una Request (Ejemplo: Crear Portfolio)

```
1. HTTP POST /api/v1/portfolios
   Body: {"name": "My BTC Portfolio", "description": "Long-term holding"}

2. PortfolioController.createPortfolio(CreatePortfolioRequest)
   ↓
3. CreatePortfolioUseCase.execute(request)
   ↓ (mapea DTO → Domain Entity)
4. Portfolio portfolio = new Portfolio(name, description, userId)
   ↓
5. portfolioRepository.save(portfolio)  // Interface del domain
   ↓
6. PortfolioRepositoryAdapter.save(portfolio)  // Implementación
   ↓ (mapea Domain Entity → JPA Entity)
7. JpaPortfolioRepository.save(portfolioEntity)
   ↓
8. PostgreSQL INSERT
   ↓
9. Return PortfolioResponse (DTO)
```

Clave: Domain nunca sabe que existe Spring, JPA, PostgreSQL. Es **pura lógica de negocio**.

📝 Roadmap de Tickets

Fase 0: Setup Inicial (1-2 días) ⚡ AUTOMATIZADO

Usa Spring Initializr + script de setup automático

- **TICKET-000**: Generar proyecto base con Spring Initializr
- **TICKET-001**: Configurar estructura de carpetas Clean Architecture

- **TICKET-002**: Setup Docker Compose (PostgreSQL + pgAdmin)
- **TICKET-003**: Configurar Flyway para migraciones
- **TICKET-004**: Setup logging (Logback)
- **TICKET-005**: Configurar perfiles (dev, test, prod)

⏱️ **Tiempo estimado: 2 horas con script automatizado** (ver sección Setup Rápido)

Fase 1: Domain Layer (3-4 días)

Sprint 1.1: Modelado de Dominio

TICKET-101: Crear entidad User (Domain)

- Crear `User.java` en `domain/entities/`
- Atributos: id, username, email, passwordHash, createdAt
- Métodos de negocio: `updateEmail()`, `isEmailValid()`
- Tests unitarios de User
- **Criterios aceptación**: Tests pasan, User es inmutable (excepto por métodos de negocio)

TICKET-102: Crear entidad Portfolio (Domain)

- Crear `Portfolio.java` en `domain/entities/`
- Atributos: id, name, description, userId, createdAt
- Métodos: `rename()`, `updateDescription()`
- Tests unitarios
- **Criterios**: Portfolio pertenece a un User (userId)

TICKET-103: Crear entidad Transaction (Domain)

- Crear `Transaction.java` en `domain/entities/`
- Atributos: id, portfolioId, crypto (BTC/ETH/etc), type (BUY/SELL), amount, pricePerUnit, timestamp
- Tests unitarios

- **Criterios**: Transaction está asociada a Portfolio

TICKET-104: Crear Value Objects

- Crear `Money.java` (amount, currency) - inmutable
- Crear `Crypto.java` (enum: BTC, ETH, ADA, etc.)
- Crear `TransactionType.java` (enum: BUY, SELL)
- Tests de inmutabilidad
- **Criterios**: Value objects son equals por valor, no por identidad

Sprint 1.2: Lógica de Negocio

TICKET-105: Crear PortfolioCalculator (Domain Service)

- Implementar `calculateBalance(List<Transaction>)` → Map<Crypto, BigDecimal>
- Implementar `calculateProfitLoss(List<Transaction>, Map<Crypto, BigDecimal> currentPrices)` → Money
- Implementar `calculateROI(List<Transaction>, Map<Crypto, BigDecimal> currentPrices)` → BigDecimal (%)
- Tests con datos reales (escenarios: solo compras, compras + ventas, pérdidas)
- **Criterios**: Lógica 100% testeada sin levantar Spring

TICKET-106: Crear Repository Interfaces (Domain Ports)

- Crear `UserRepository.java` (interface en domain)
- Métodos: `save()`, `findById()`, `findByEmail()`, `delete()`
- Crear `PortfolioRepository.java`
- Métodos: `save()`, `findById()`, `findByUserId()`, `delete()`
- Crear `TransactionRepository.java`
- Métodos: `save()`, `findByPortfolioId()`, `delete()`
- **Criterios**: Son interfaces puras, sin anotaciones de Spring

TICKET-107: Crear Domain Exceptions

- `UserNotFoundException.java`

- `PortfolioNotFoundException.java`
- `InsufficientFundsException.java` (para ventas que exceden balance)
- `InvalidTransactionException.java`
- **Criterios**: Extender RuntimeException, mensajes descriptivos

---

Fase 2: Application Layer (4-5 días)

Sprint 2.1: DTOs y Mappers

TICKET-201: Crear DTOs de Request

- `RegisterUserRequest.java` (username, email, password)
- `LoginRequest.java` (email, password)
- `CreatePortfolioRequest.java` (name, description)
- `AddTransactionRequest.java` (portfolioId, crypto, type, amount, pricePerUnit)
- Validaciones con `@NotNull`, `@Email`, `@Size`, `@Positive`
- **Criterios**: Todas las validaciones Bean Validation presentes

TICKET-202: Crear DTOs de Response

- `AuthResponse.java` (token, userId, username)
- `PortfolioResponse.java` (id, name, description, createdAt)
- `PortfolioSummaryResponse.java` (portfolio + balance + P&L + ROI)
- `TransactionResponse.java` (id, crypto, type, amount, pricePerUnit, timestamp)
- **Criterios**: DTOs son POJOs, solo getters

TICKET-203: Implementar Mappers con MapStruct

- Crear `UserMapper.java` (Entity ↔ DTO)
- Crear `PortfolioMapper.java`
- Crear `TransactionMapper.java`
- Configurar MapStruct en `pom.xml`

- Tests de mapeo (ida y vuelta)
- **Criterios**: Mappers generados automáticamente por MapStruct

Sprint 2.2: Use Cases de Auth

TICKET-204: Implementar RegisterUserUseCase

- Validar que email no exista
- Hashear password con BCrypt
- Guardar User en repository
- Return AuthResponse con JWT
- Tests: usuario duplicado, validaciones, happy path
- **Criterios**: Password nunca se almacena en texto plano

TICKET-205: Implementar LoginUserUseCase

- Buscar user por email
- Verificar password con BCrypt
- Generar JWT
- Return AuthResponse
- Tests: credenciales inválidas, usuario no existe, happy path
- **Criterios**: JWT contiene userId y roles

Sprint 2.3: Use Cases de Portfolio

TICKET-206: Implementar CreatePortfolioUseCase

- Validar que userId existe
- Crear Portfolio
- Guardar en repository
- Return PortfolioResponse
- Tests con mocks de repository
- **Criterios**: Un usuario puede crear múltiples portfolios

TICKET-207: Implementar GetPortfolioUseCase

- Buscar portfolio por ID
- Verificar ownership (portfolio.userId == requestUserId)
- Return PortfolioResponse
- Tests: portfolio no encontrado, no autorizado, happy path
- **Criterios**: Un usuario NO puede ver portfolios de otros

TICKET-208: Implementar ListUserPortfoliosUseCase

- Buscar todos los portfolios de un usuario
- Return List<PortfolioResponse>
- Tests con 0, 1, N portfolios
- **Criterios**: Solo devuelve portfolios del usuario autenticado

TICKET-209: Implementar UpdatePortfolioUseCase

- Buscar portfolio, verificar ownership
- Actualizar name/description
- Guardar
- Return PortfolioResponse actualizado
- Tests
- **Criterios**: Solo owner puede actualizar

TICKET-210: Implementar DeletePortfolioUseCase

- Buscar portfolio, verificar ownership
- Eliminar (cascade transactions si las hay)
- Tests
- **Criterios**: Eliminar portfolio elimina todas sus transactions

Sprint 2.4: Use Cases de Transaction

TICKET-211: Implementar AddTransactionUseCase

- Validar que portfolio existe y pertenece al user
- Crear Transaction

- Si es SELL, validar balance suficiente con PortfolioCalculator
- Guardar transaction
- Return TransactionResponse
- Tests: venta sin fondos, compra válida, sell válido
- **Criterios**: No se puede vender más crypto del que se tiene

TICKET-212: Implementar GetTransactionHistoryUseCase

- Buscar portfolio, verificar ownership
- Buscar todas las transactions del portfolio (ordenadas por timestamp desc)
- Return List<TransactionResponse>
- Tests
- **Criterios**: Devuelve transacciones más recientes primero

TICKET-213: Implementar DeleteTransactionUseCase

- Buscar transaction, verificar ownership del portfolio asociado
- Eliminar transaction
- Tests
- **Criterios**: Solo owner del portfolio puede eliminar transactions

TICKET-214: Implementar GetPortfolioSummaryUseCase

- Buscar portfolio + todas sus transactions
- Calcular balance actual con PortfolioCalculator
- Obtener precios actuales de CryptoPriceProvider (port)
- Calcular P&L y ROI
- Return PortfolioSummaryResponse
- Tests con precios mockeados
- **Criterios**: Muestra balance, total invertido, P&L, ROI%

Fase 3: Infrastructure Layer (5-6 días)

Sprint 3.1: Persistence con JPA

TICKET-301: Crear JPA Entities

- `UserEntity.java` con anotaciones JPA (@Entity, @Table, @Id, @GeneratedValue)
- `PortfolioEntity.java` con @ManyToOne a User
- `TransactionEntity.java` con @ManyToOne a Portfolio
- Configurar relaciones bidireccionales
- **Criterios**: Entities tienen equivalencia 1:1 con Domain entities

TICKET-302: Crear Spring Data Repositories

- `JpaUserRepository.java` extends JpaRepository
- `JpaPortfolioRepository.java`
- `JpaTransactionRepository.java`
- Métodos custom: `findByEmail()`, `findByUserId()`, `findByPortfolioId()`
- **Criterios**: Usar naming conventions de Spring Data para queries

TICKET-303: Implementar Repository Adapters

- `UserRepositoryAdapter.java` implementa domain `UserRepository`
- Mapea Domain Entity ↔ JPA Entity
- Inyecta JpaUserRepository
- Implementar todos los métodos del port
- Tests con H2 in-memory
- **Criterios**: Adapters aíslan Domain de JPA

TICKET-304: Implementar PortfolioRepositoryAdapter

- Implementa domain `PortfolioRepository`
- Mapeo JPA ↔ Domain
- Tests
- **Criterios**: Mismo patrón que UserRepositoryAdapter

TICKET-305: Implementar TransactionRepositoryAdapter

- Implementa domain `TransactionRepository`
- Tests
- **Criterios**: Mismo patrón

TICKET-306: Configurar Flyway Migrations

- Crear `V1__create_users_table.sql`
- Crear `V2__create_portfolios_table.sql`
- Crear `V3__create_transactions_table.sql`
- Crear índices en foreign keys
- **Criterios**: Migrations ejecutan correctamente en PostgreSQL

Sprint 3.2: Spring Security + JWT

TICKET-307: Implementar JwtTokenProvider

- Métodos: `generateToken(userId, roles)`, `validateToken(token)`, `getUserIdFromToken(token)`
- Usar librería `io.jsonwebtoken:jjwt`
- Configurar secret y expiration en `application.yml`
- Tests unitarios
- **Criterios**: Tokens expiran en 24h, firma HMAC256

TICKET-308: Implementar UserDetailsServiceImpl

- Implementa `UserDetailsService` de Spring Security
- Carga User desde database
- Return `UserDetails` con roles
- **Criterios**: Integrado con Spring Security

TICKET-309: Crear JwtAuthenticationFilter

- Filtro que intercepta requests
- Extrae JWT del header `Authorization: Bearer <token>`
- Valida token con JwtTokenProvider

- Setea Authentication en SecurityContext
- **Criterios**: Requests sin token → 401, token inválido → 401

TICKET-310: Configurar SecurityConfig

- Configurar SecurityFilterChain
- Endpoints públicos: `/api/v1/auth/**`, `/swagger-ui/**`, `/v3/api-docs/**`
- Endpoints protegidos: `/api/v1/portfolios/**`, `/api/v1/transactions/**`
- Registrar JwtAuthenticationFilter
- CORS habilitado
- **Criterios**: Auth funciona end-to-end

Sprint 3.3: REST Controllers

TICKET-311: Implementar AuthController

- `POST /api/v1/auth/register` → RegisterUserUseCase
- `POST /api/v1/auth/login` → LoginUserUseCase
- Validación de DTOs con `@Valid`
- Manejo de errores (400 si validación falla, 409 si user existe)
- **Criterios**: Devuelve JWT válido en respuesta

TICKET-312: Implementar PortfolioController

- `POST /api/v1/portfolios` → CreatePortfolioUseCase
- `GET /api/v1/portfolios` → ListUserPortfoliosUseCase
- `GET /api/v1/portfolios/{id}` → GetPortfolioUseCase
- `PUT /api/v1/portfolios/{id}` → UpdatePortfolioUseCase
- `DELETE /api/v1/portfolios/{id}` → DeletePortfolioUseCase
- `GET /api/v1/portfolios/{id}/summary` → GetPortfolioSummaryUseCase
- Autorización: verificar userId del token
- **Criterios**: CRUD completo funcional

TICKET-313: Implementar TransactionController

- `POST /api/v1/transactions` → AddTransactionUseCase
- `GET /api/v1/portfolios/{portfolioId}/transactions` → GetTransactionHistoryUseCase
- `DELETE /api/v1/transactions/{id}` → DeleteTransactionUseCase
- **Criterios**: Transacciones asociadas a portfolios

Sprint 3.4: Manejo de Errores y Validación

TICKET-314: Implementar GlobalExceptionHandler

- `@ControllerAdvice` con `@ExceptionHandler`
- Mapear domain exceptions a HTTP status:
  - `NotFoundException` → 404
  - `ValidationException` → 400
  - `UnauthorizedException` → 401
  - `InsufficientFundsException` → 400
- Return `ErrorResponse` (timestamp, status, message, path)
- **Criterios**: Errores consistentes en toda la API

TICKET-315: Configurar Bean Validation

- Habilitar `@Valid` en controllers
- Custom validators si es necesario (ej: validar crypto code)
- Tests de validación en controllers
- **Criterios**: Requests inválidos devuelven 400 con detalles

Sprint 3.5: Integración Externa (CoinGecko API)

TICKET-316: Crear CryptoPriceProvider Port (Application)

- Interface `CryptoPriceProvider.java` en `application/ports/`
- Método: `getCurrentPrice(Crypto crypto) → BigDecimal`
- **Criterios**: Port define contrato, no implementación

TICKET-317: Implementar CoinGeckoPriceProvider (Infrastructure)

- Implementa `CryptoPriceProvider`
- Consume CoinGecko API pública (https://api.coingecko.com/api/v3/simple/price)
- Usa `RestTemplate` o `WebClient`
- Cachear resultados (5 min TTL) con `@Cacheable` si usamos Redis, o in-memory
- Tests con WireMock (mock de API externa)
- **Criterios**: Si API falla, usar precios default o lanzar exception

---

Fase 4: Testing (3-4 días)

TICKET-401: Tests Unitarios de Domain

- Cobertura >90% de `domain/` package
- Tests de entidades, value objects, services
- Mock-free (domain no tiene dependencias)
- **Criterios**: Tests rápidos (<100ms total)

TICKET-402: Tests Unitarios de Application (Use Cases)

- Mock de repositories con Mockito
- Tests de todos los use cases
- Cobertura >85%
- **Criterios**: Verificar interacciones con mocks

TICKET-403: Tests de Integración con Testcontainers

- Setup Testcontainers PostgreSQL
- Tests de Repository Adapters (guardar, buscar, actualizar, eliminar)
- Tests de transacciones completas (crear user → portfolio → transaction)
- **Criterios**: Tests usan PostgreSQL real en Docker

TICKET-404: Tests E2E de API con REST Assured

- `AuthControllerE2ETest` (register, login, JWT)
- `PortfolioControllerE2ETest` (CRUD completo)

- `TransactionControllerE2ETest` (añadir, listar)
- Tests de autorización (intentar acceder a portfolio ajeno)
- **Criterios**: Tests cubren happy path + edge cases

TICKET-405: Configurar JaCoCo para Code Coverage

- Plugin JaCoCo en `pom.xml`
- Generar reporte HTML de cobertura
- Forzar mínimo 75% cobertura (build falla si <75%)
- **Criterios**: Reporte visible en `target/site/jacoco/index.html`

Fase 5: Documentación y DevOps (2-3 días)

TICKET-501: Configurar OpenAPI/Swagger

- Agregar `springdoc-openapi-starter-webmvc-ui`
- Configurar metadata en `OpenAPIConfig.java` (título, versión, descripción)
- Añadir `@Operation`, `@ApiResponse` en controllers
- **Criterios**: Swagger UI en `http://localhost:8080/swagger-ui.html`

TICKET-502: Dockerizar aplicación

- Crear `Dockerfile` multi-stage (build + runtime)
- Crear `docker-compose.yml` (app + PostgreSQL + pgAdmin)
- Variables de entorno para DB connection
- **Criterios**: `docker-compose up` levanta toda la app

TICKET-503: Configurar GitHub Actions (CI)

- Workflow: build → tests → code coverage
- Ejecutar en push a `main` y en PRs
- Subir reporte de cobertura como artifact
- **Criterios**: Pipeline verde en GitHub

TICKET-504: Escribir README completo

- Descripción del proyecto
- Tech stack
- Diagrama de arquitectura (draw.io)
- Quick start con Docker
- Endpoints de API
- Estructura de carpetas
- Contribuir
- **Criterios**: README profesional, con badges de CI, cobertura

TICKET-505: Deployment en Railway/Render

- Crear cuenta en Railway
- Deploy backend con PostgreSQL
- Configurar variables de entorno
- Test de endpoints públicos
- **Criterios**: API accesible en https://cryptofolio-api.railway.app

---

Fase 6 (Opcional): Frontend React

TICKET-601: Setup proyecto React con Vite + TypeScript

- `npm create vite@latest cryptofolio-frontend --template react-ts`
- Configurar Tailwind CSS
- Instalar shadcn/ui
- **Criterios**: App corre en `http://localhost:5173`

TICKET-602: Implementar autenticación (Login/Register)

- Formularios con React Hook Form + Zod
- Llamadas a API con TanStack Query
- Almacenar JWT en localStorage
- Protected routes con React Router

- **Criterios**: Login funcional, redirect a dashboard

TICKET-603: Dashboard de portfolios

- Listar portfolios del user
- Botón "Create Portfolio"
- Ver detalles de portfolio (balance, P&L, ROI)
- **Criterios**: UI responsive, datos reales de API

TICKET-604: Gestión de transacciones

- Modal "Add Transaction" (BUY/SELL)
- Lista de transacciones de portfolio
- Eliminar transacción
- **Criterios**: Operaciones CRUD funcionan

TICKET-605: Gráficos con Recharts

- Gráfico de balance histórico
- Distribución de crypto (pie chart)
- P&L timeline
- **Criterios**: Visualizaciones claras y útiles

⚡ Setup Rápido Inicial (Automatización)

Opción 1: Spring Initializr Web

1. **Ir a https://start.spring.io/**
2. **Configuración:**
    - Project: Maven
    - Language: Java
    - Spring Boot: 3.2.4 (última estable)
    - Packaging: Jar
    - Java: 17

- Group: com.cryptofolio
- Artifact: cryptofolio-api
- Name: CryptoFolio API
- Package name: com.cryptofolio

3. **Dependencies (seleccionar):**
   - Spring Web
   - Spring Security
   - Spring Data JPA
   - PostgreSQL Driver
   - Validation
   - Lombok
   - Spring Boot DevTools
   - Spring Boot Actuator
4. **Generate → Descargar ZIP → Extraer**

Opción 2: Script de Setup Automatizado (RECOMENDADO)

Crea este script `setup-cryptofolio.sh`:

```
#!/bin/bash

echo "🚀 Setting up CryptoFolio API project..."

# 1. Generar proyecto con Spring Initializr (CLI)
curl https://start.spring.io/starter.zip \
  -d dependencies=web,security,data-jpa,postgresql,validation,lombok,devtools,act
  -d type=maven-project \
  -d language=java \
  -d bootVersion=3.2.4 \
  -d baseDir=cryptofolio-api \
  -d groupId=com.cryptofolio \
  -d artifactId=cryptofolio-api \
  -d name=CryptoFolio \
  -d packageName=com.cryptofolio \
  -d javaVersion=17 \
  -o cryptofolio-api.zip
```

```
unzip cryptofolio-api.zip
cd cryptofolio-api
rm ../cryptofolio-api.zip

# 2. Crear estructura Clean Architecture
mkdir -p src/main/java/com/cryptofolio/{domain/{entities,valueobjects,repositorie

mkdir -p src/main/java/com/cryptofolio/infrastructure/persistence/{entities,repos

# 3. Crear estructura de tests
mkdir -p src/test/java/com/cryptofolio/{domain,application,infrastructure}

# 4. Agregar dependencias adicionales al pom.xml
cat >> pom.xml << 'EOF'
                <!-- JWT -->
                <dependency>
                        <groupId>io.jsonwebtoken</groupId>
                        <artifactId>jjwt-api</artifactId>
                        <version>0.12.5</version>
                </dependency>
                <dependency>
                        <groupId>io.jsonwebtoken</groupId>
                        <artifactId>jjwt-impl</artifactId>
                        <version>0.12.5</version>
                        <scope>runtime</scope>
                </dependency>
                <dependency>
                        <groupId>io.jsonwebtoken</groupId>
                        <artifactId>jjwt-jackson</artifactId>
                        <version>0.12.5</version>
                        <scope>runtime</scope>
                </dependency>

                <!-- OpenAPI/Swagger -->
                <dependency>
                        <groupId>org.springdoc</groupId>
                        <artifactId>springdoc-openapi-starter-webmvc-ui</artifact
                        <version>2.3.0</version>
                </dependency>

                <!-- MapStruct -->
                <dependency>
                        <groupId>org.mapstruct</groupId>
                        <artifactId>mapstruct</artifactId>
                        <version>1.5.5.Final</version>
                </dependency>
```

```
<dependency>
        <groupId>org.mapstruct</groupId>
        <artifactId>mapstruct-processor</artifactId>
        <version>1.5.5.Final</version>
        <scope>provided</scope>
</dependency>

<!-- Flyway -->
<dependency>
        <groupId>org.flywaydb</groupId>
        <artifactId>flyway-core</artifactId>
</dependency>
<dependency>
        <groupId>org.flywaydb</groupId>
        <artifactId>flyway-database-postgresql</artifactId>
</dependency>

<!-- Testcontainers -->
<dependency>
        <groupId>org.testcontainers</groupId>
        <artifactId>testcontainers</artifactId>
        <version>1.19.7</version>
        <scope>test</scope>
</dependency>
<dependency>
        <groupId>org.testcontainers</groupId>
        <artifactId>postgresql</artifactId>
        <version>1.19.7</version>
        <scope>test</scope>
</dependency>
<dependency>
        <groupId>org.testcontainers</groupId>
        <artifactId>junit-jupiter</artifactId>
        <version>1.19.7</version>
        <scope>test</scope>
</dependency>

<!-- REST Assured (E2E tests) -->
<dependency>
        <groupId>io.rest-assured</groupId>
        <artifactId>rest-assured</artifactId>
        <version>5.4.0</version>
        <scope>test</scope>
</dependency>

<!-- AssertJ -->
<dependency>
```

```
                <groupId>org.assertj</groupId>
                <artifactId>assertj-core</artifactId>
                <version>3.25.3</version>
                <scope>test</scope>
        </dependency>
</dependencies>

<build>
        <plugins>
                <!-- JaCoCo -->
                <plugin>
                        <groupId>org.jacoco</groupId>
                        <artifactId>jacoco-maven-plugin</artifactId>
                        <version>0.8.11</version>
                        <executions>
                                <execution>
                                        <goals>
                                                <goal>prepare-agent</goal
                                        </goals>
                                </execution>
                                <execution>
                                        <id>report</id>
                                        <phase>test</phase>
                                        <goals>
                                                <goal>report</goal>
                                        </goals>
                                </execution>
                                <execution>
                                        <id>check</id>
                                        <goals>
                                                <goal>check</goal>
                                        </goals>
                                        <configuration>
                                                <rules>
                                                        <rule>
                                                                <element>
                                                                <limits>
                                                                        <


                                                                        <
                                                                </limits>
                                                        </rule>
                                                </rules>
                                        </configuration>
                                </execution>
```

```
                                </executions>
                        </plugin>
                </plugins>
        </build>
EOF

# 5. Crear application.yml
cat > src/main/resources/application.yml << 'EOF'
spring:
  application:
    name: cryptofolio-api

  profiles:
    active: dev

---
# Development Profile
spring:
  config:
    activate:
      on-profile: dev

  datasource:
    url: jdbc:postgresql://localhost:5432/cryptofolio_dev
    username: postgres
    password: postgres
    driver-class-name: org.postgresql.Driver

  jpa:
    hibernate:
      ddl-auto: validate
    show-sql: true
    properties:
      hibernate:
        format_sql: true

  flyway:
    enabled: true
    baseline-on-migrate: true

jwt:
  secret: your-secret-key-change-in-production-min-256-bits
  expiration: 86400000 # 24 hours

logging:
  level:
    com.cryptofolio: DEBUG
```

```
    org.springframework.security: DEBUG

---
# Test Profile
spring:
  config:
    activate:
      on-profile: test

  datasource:
    url: jdbc:h2:mem:testdb
    driver-class-name: org.h2.Driver

  jpa:
    hibernate:
      ddl-auto: create-drop

  flyway:
    enabled: false

---
# Production Profile
spring:
  config:
    activate:
      on-profile: prod

  datasource:
    url: ${DATABASE_URL}
    username: ${DATABASE_USERNAME}
    password: ${DATABASE_PASSWORD}

  jpa:
    hibernate:
      ddl-auto: validate
    show-sql: false

jwt:
  secret: ${JWT_SECRET}
  expiration: 86400000

logging:
  level:
    com.cryptofolio: INFO
EOF

# 6. Crear docker-compose.yml
```

```
cat > docker-compose.yml << 'EOF'
version: '3.8'

services:
  postgres:
    image: postgres:16-alpine
    container_name: cryptofolio-db
    environment:
      POSTGRES_DB: cryptofolio_dev
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 10s
      timeout: 5s
      retries: 5

  pgadmin:
    image: dpage/pgadmin4:latest
    container_name: cryptofolio-pgadmin
    environment:
      PGADMIN_DEFAULT_EMAIL: admin@cryptofolio.com
      PGADMIN_DEFAULT_PASSWORD: admin
    ports:
      - "5050:80"
    depends_on:
      - postgres

volumes:
  postgres_data:
EOF

# 7. Crear migrations iniciales
mkdir -p src/main/resources/db/migration

cat > src/main/resources/db/migration/V1__create_users_table.sql << 'EOF'
CREATE TABLE users (
    id BIGSERIAL PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    email VARCHAR(100) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE INDEX idx_users_email ON users(email);
EOF

cat > src/main/resources/db/migration/V2__create_portfolios_table.sql << 'EOF'
CREATE TABLE portfolios (
    id BIGSERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    description TEXT,
    user_id BIGINT NOT NULL,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_portfolio_user FOREIGN KEY (user_id) REFERENCES users(id) ON DE
);

CREATE INDEX idx_portfolios_user_id ON portfolios(user_id);
EOF

cat > src/main/resources/db/migration/V3__create_transactions_table.sql << 'EOF'
CREATE TABLE transactions (
    id BIGSERIAL PRIMARY KEY,
    portfolio_id BIGINT NOT NULL,
    crypto VARCHAR(10) NOT NULL,
    transaction_type VARCHAR(4) NOT NULL CHECK (transaction_type IN ('BUY', 'SELL
    amount DECIMAL(20, 8) NOT NULL,
    price_per_unit DECIMAL(20, 2) NOT NULL,
    timestamp TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_transaction_portfolio FOREIGN KEY (portfolio_id) REFERENCES por
);

CREATE INDEX idx_transactions_portfolio_id ON transactions(portfolio_id);
CREATE INDEX idx_transactions_timestamp ON transactions(timestamp DESC);
EOF

# 8. Crear .gitignore
cat > .gitignore << 'EOF'
target/
!.mvn/wrapper/maven-wrapper.jar
.mvn/wrapper/maven-wrapper.properties

### STS ###
.apt_generated
.classpath
.factorypath
.project
.settings
.springBeans
.sts4-cache
```

```
### IntelliJ IDEA ###
.idea
*.iws
*.iml
*.ipr

### VS Code ###
.vscode/

### Logs ###
*.log

### OS ###
.DS_Store
Thumbs.db
EOF

# 9. Crear README básico
cat > README.md << 'EOF'
# 🌑 CryptoFolio API

Portfolio de criptomonedas con Clean Architecture.

## 🚀 Quick Start

```bash
# 1. Levantar PostgreSQL
docker-compose up -d

# 2. Run app
./mvnw spring-boot:run

# 3. Swagger UI
open http://localhost:8080/swagger-ui.html
```
```

📊 Tech Stack

- Java 17
- Spring Boot 3.2.x
- PostgreSQL 16
- Clean Architecture

- JWT Authentication

🧪 Tests

```
./mvnw test
./mvnw jacoco:report
open target/site/jacoco/index.html
```

EOF

echo "✅ Setup complete!" echo "📁 Project created in: cryptofolio-api/" echo "" echo "Next steps:" echo "1. cd cryptofolio-api" echo "2. docker-compose up -d" echo "3. ./mvnw spring-boot:run" echo "4. Open http://localhost:8080/swagger-ui.html"

```
**Ejecutar:**
```bash
chmod +x setup-cryptofolio.sh
./setup-cryptofolio.sh
```
```

⏱️ **Tiempo total: 5-10 minutos**

📚 Recursos Adicionales

Libros/Guías

- Clean Architecture (Robert C. Martin)
- Domain-Driven Design Distilled (Vaughn Vernon)
- Spring Boot in Action (Craig Walls)

Tutoriales

- Baeldung - Spring Security with JWT
- Clean Architecture with Spring Boot

Tools

- Spring Initializr
- Testcontainers

- MapStruct

✅ Checklist Final

Antes de considerar el proyecto “terminado”:

- Todos los tests pasan (>75% cobertura)
- Swagger UI muestra todos los endpoints
- Docker Compose funciona (`docker-compose up`)
- CI pipeline en GitHub Actions está verde
- README completo con diagramas
- Deployed en Railway/Render
- No hay credenciales hardcodeadas
- Logs configurados correctamente
- Manejo de errores consistente
- Code review propio (refactorizar antes de pushear)

Fecha creación: 2026-04-03 **Última actualización:** 2026-04-03 **Autor:** Cristian Chozas **Versión:** 1.0